

Basics4AI — Solution Architecture Document

Version 1.0 — 2026-07-11. Describes the system as deployed to production as of this date.

1. Overview

Basics4AI is a Streamlit-based web platform for teaching AI literacy to school-age participants (10–14 years old) through a sequence of instructional modules, each paired with research instruments (content-knowledge assessments, motivation/self-efficacy surveys, and open-ended reflections). It exists to serve two audiences at once:

- **Participants** (students) work through modules in order, unlocking each one as they complete the previous.
- **Researchers and teachers** use a built-in analytics dashboard to monitor progress and run statistical analyses (descriptive statistics, inferential tests, Item Response Theory, LLM-assisted qualitative analysis, and a custom Competency Progression Index) directly against the collected data — no separate export/import step into R, SPSS, or similar tools is required for most analyses.

The platform is currently mid-pilot with a live cohort, hosted on Railway, with all source in a public GitHub repository.

2. Technology Stack

Layer	Technology
Application framework	Streamlit (Python), single-process web app
Database	SQLite in WAL mode (three separate .db files — see §5)
Hosting	Railway (Docker-based), two environments: production and test
Containerization	Docker, multi-stage build (Dockerfile)
Analytics / stats	pandas, scipy, pingouin (inferential stats), girth (Item Response Theory, pure Python)
Qualitative analysis	LLM provider APIs — OpenAI, Anthropic, Google Gemini
Reporting	reportlab (PDF generation), python-docx
Scheduled maintenance	GitHub Actions (nightly restart workaround — see §7.3)

SQLite was chosen for simplicity of a single-researcher-operated deployment; a future migration to PostgreSQL is on the post-pilot roadmap (not part of this document's scope).

3. Layered Architecture

The codebase separates into five layers, each with a distinct responsibility. Files in one layer generally do not reach into another layer's internals directly — for example, `core/progress_engine.py` never imports Streamlit, and the module content layer never opens a database connection directly.

3.1 Presentation layer — `streamlit_app/`

The only layer that imports Streamlit and renders UI.

- **`streamlit_app/app.py`** — the application entry point (Dockerfile runs `streamlit run streamlit_app/app.py`). On every process start it: forces UTF-8 console encoding, loads environment config, registers all active modules into the module registry (§3.3), initializes both databases' schemas, seeds the super-admin account (idempotently — see §8.1), and starts the auto-backup scheduler. On every page load it authenticates the user (or shows registration/login), resolves their role, and dispatches to the appropriate dashboard.
- **`streamlit_app/dashboards/student_dashboard.py`** — a single view listing all active modules in order, with a status badge per module (locked / available / completed) derived live from the database — no cached or stored “unlock state” exists anywhere.
- **`streamlit_app/dashboards/teacher_dashboard.py`** — the analytics console. Loads a cached “canonical dataset” (5-minute TTL, with a manual refresh button) and offers six sections: Basic Statistics, Inferential Statistics, IRT Analysis, LLM Analysis, Competency Progression Index (CPI), and Report Generation (PDF export).
- **`streamlit_app/dashboards/admin_dashboard.py`** — administrative console with six tabs: Pending Approvals (super-admin only), User Management, Data Management, Research Operations, System Operations, and Diagnostics.
- **`streamlit_app/surveys/`** — YAML definitions for survey/scoring instruments. Some are per-module (e.g. `module1_content_mcq_assessment_scoring.yaml`), others are shared study-wide instruments administered identically across every module (e.g. `b4ai_sccces_survey.yaml`, `b4ai_sims_survey.yaml`) — editing a shared survey file changes that instrument for every module at once, which is an intentional design (see the Module Lifecycle Guide, §6).

3.2 Business logic layer — `core/`

- **`core/progress_engine.py`** — the single source of truth for “is this module unlocked.” Purely positional: the first module in registry order is always unlocked; every subsequent module unlocks once the previous one is marked complete in the

database. No caching, no Streamlit dependency — a deliberate design choice so this logic can be tested and trusted independently of the UI.

- **core/db_utils.py** — the database abstraction layer. Owns SQLite connection setup (WAL mode, busy_timeout), schema creation for responses.db's four core tables, and a set of completion-tracking helpers. Also contains an unused PostgreSQL connection branch (DB_TYPE=postgres) that would fail at runtime today since psycopg2 isn't installed — dormant scaffolding for the future database migration, not currently reachable.
- **core/submission_engine.py** — receives a completed instrument's answers from a module's render function, stores raw responses, triggers scoring, and marks the instrument complete.
- **core/scoring_engine.py** — computes assessment/survey scores from raw responses.
- **core/admin/** — backend logic for every admin dashboard action:
 - **system_service.py** — backup/restore/clone of the databases, plus the auto-backup scheduler (with a startup catch-up check so a scheduler reset by a restart doesn't silently skip backups).
 - **data_service.py** — targeted data-reset operations (single student, single instrument, or the intersection of both), all routed through core/db_utils.py's connection helper so they stay consistent with the rest of the app.
 - **audit_logger.py** — records every admin action to an admin_logs table for accountability.
- **core/analytics/** — the research pipeline, organized by analysis type:
 - **datasets/** — assembles the “canonical dataset” the Teacher Dashboard and every analysis module consumes.
 - **descriptive/, inferential/, irt/** — statistical analysis engines.
 - **cpi/** — the Competency Progression Index engine (writes its own results tables).
 - **llm/** — LLM-assisted qualitative analysis (inductive and deductive thematic analysis pipelines), with its own result-caching database (§5.3).

3.3 Domain content layer — `modules/` + `content_dev/`

This layer defines *what* each module teaches and asks, independent of *how* it's rendered or stored.

- **modules/definitions/moduleN_definition.py** — one file per module, each exporting a MODULE_DEFINITION dictionary describing: metadata (title, order, active/disabled status), pedagogy (learning objectives, estimated time), instruments (which assessments and surveys this module includes, and where their content lives), evaluation rules, UI ordering, and constraints (max attempts, time limits).

- **modules/registry/** — the auto-discovery mechanism:
 - `discover.py` scans `modules/definitions/` for every `*_definition.py` file, validates it, and — critically — skips any module whose status is not "active". This is the exact mechanism used to hide Module 7 for the current pilot (see the Module Lifecycle Guide).
 - `module_registry.py` holds the registered, ordered list of modules the rest of the app queries.
 - `adapters.py` normalizes each raw definition and dynamically wires in that module's actual `render()` function.
 - `register_modules.py` chains discovery → adaptation → registration; this is what `app.py` calls once per session at boot.
- **modules/resolution/learning_unit_resolver.py** — a read-only, deterministic ordering layer that `progress_engine.py` builds its unlock logic on top of.
- **content_dev/** — the actual question banks (`moduleN_question_bank.json`) and shared reference data (learning objectives, construct definitions) that module definitions point to.

3.4 Data layer — three SQLite databases

See §5 for full schema detail. In brief: `responses.db` holds all participant-generated data (answers, scores, completion flags) plus every analytics engine's result tables; `users.db` holds accounts, login-attempt history, cohorts, and the admin audit log; `research.db` holds a small LLM-result cache, keyed by content hash.

3.5 Authentication layer — auth/

- **user_manager.py** — the live, actually-used auth module: registration, authentication, role resolution, the pending-approval workflow, and password management.
- **login_security.py** — independent, DB-backed login-lockout tracking (3 failed attempts locks an account for 5 minutes by default, both configurable), plus an admin override for time-boxed pilot sessions.
- **password_utils.py** — present in the codebase but **not used anywhere** (see §8.2).

4. User Roles

Roles are plain strings, not a formal enum, and are checked ad hoc throughout the presentation layer.

Role	Stored as	Can do
Student	<code>role='student'</code>	Work through active modules in unlocked order; answer assessments,

Role	Stored as	Can do
Teacher	role='teacher'	surveys, and reflections. Full read access to the Teacher Dashboard: all analytics, reports, and dataset views. No write/admin actions.
Admin	role='admin'	Everything a teacher can do, plus User Management, Data Management (resets), Research Operations, System Operations (backups), and Diagnostics.
Super-admin	role='admin' + is_super_admin=1 flag	Everything an admin can do, plus the Pending Approvals tab (approve/reject new registrations). There is exactly one super-admin account, seeded at boot (see §8.1). This is a derived pseudo-role, not a distinct database value — it exists because the users table's role column has a hard CHECK constraint that only allows student/teacher/admin.

5. Data Layer Detail

5.1 responses.db

Table	Purpose
responses	One row per question answered — the rawest layer of participant data.
survey_scores	One computed score per user per survey instrument.
assessment_scores	One computed score per user per MCQ assessment.
completions	One row per (user, module, instrument) marking it done — this table is the actual

Table	Purpose
transcripts, ita_runs/ita_results, dta_runs/dta_results/dta_lo_results	source of truth the unlock logic reads. LLM qualitative-analysis pipeline inputs/outputs.
cpi_runs, cpi_qual_scores, cpi_summary	Competency Progression Index engine outputs.

5.2 users.db

Table	Purpose
users	Accounts — username, hashed password, role, cohort, approval status, super-admin flag.
login_attempts	Every login attempt (success/failure, timestamp) — powers the lockout mechanism.
cohorts	Named participant groupings.
admin_logs	Audit trail of every admin action.
restore_log	Created on first use of the restore feature — before/after row counts for every restore, for verification.

5.3 research.db

A single table, `llm_result_cache`, keyed by a content hash — avoids re-paying for identical LLM analysis calls. See §8.4 for a configuration inconsistency around this database’s path.

6. Data Flow

Registration → Approval → Login → Dashboard routing → Module rendering → Submission → Scoring → Analytics

1. **Registration** — a new user signs up; account is created with `status='pending'`.
2. **Approval** — the super-admin approves or rejects the account from the Pending Approvals tab.
3. **Login** — checked against the lockout mechanism first, then authenticated; role and approval status gate access.
4. **Dashboard routing** — the app dispatches to the Student, Teacher, or Admin dashboard based on role.
5. **Module list / unlock** (students only) — the registry’s ordered module list is cross-checked against the completions table, live, on every render.

6. **Module rendering** — clicking a module invokes that module's own `render()` function, which reads what's already been completed and displays the appropriate assessment/survey/reflection UI.
 7. **Submission** — answers are written to responses, scored (writing to `survey_scores/assessment_scores`), and the instrument is marked in `completions` — which is what makes the *next* module unlock on the participant's next view.
 8. **Analytics** — the Teacher Dashboard loads a canonical, cross-instrument dataset assembled from all of the above, then feeds it into whichever analysis engine the teacher/researcher selects.
-

7. Module Lifecycle & Deployment

7.1 Module lifecycle

Modules are added, hidden, or re-enabled entirely through the status field in their definition file (§3.3) — no other code changes are required for a hide/re-enable. The full procedure (hiding, re-enabling, adding a brand-new module, replacing content) is documented separately in `docs/module-lifecycle-guide.md`, which should be treated as the canonical reference for this procedure rather than duplicated here.

7.2 Deployment environments

Railway hosts two environments under one project, sharing a service definition but with separate volumes and environment variables:

- **production** — the live pilot deployment. Auto-deploys whenever `origin/main` on GitHub is pushed.
- **test** — a disposable validation environment with its own isolated volume, used to verify changes against realistic data before they reach production. Also tracks `origin/main` for auto-deploy (there is currently no separate test-branch auto-deploy wiring; deploying arbitrary branches there requires a manual railway up).

The standard change-safety workflow: build and test locally → push to a disposable test git branch → merge into main only after both local and live-test-environment verification → push to main, which deploys to production automatically. Because both Railway environments' services are configured identically and only differ by volume/env-vars, a push to main deploys to **both** environments simultaneously.

7.3 Known operational issue: Streamlit websocket/session-leak workaround

There is an upstream, unresolved bug in Streamlit itself (tracked publicly as `streamlit/streamlit#8901`) involving websocket/session memory accumulation over long-running processes. Rather than working around it in application code, the mitigation is operational: `.github/workflows/scheduled-restart.yml` triggers a Railway redeploy of the production service every night at 07:00 UTC via a GitHub Actions cron job, using a

RAILWAY_TOKEN secret scoped specifically to the production environment (verified empirically during this pilot by observing production’s deployment ID change on a manual trigger while test’s stayed identical). This keeps the process young enough that the leak never becomes user-visible, without requiring a fix to Streamlit itself.

One interaction to be aware of: the nightly restart also resets two in-memory, process-local settings back to their safe defaults — the auto-backup scheduler’s internal timer (mitigated by a startup catch-up check, §3.2) and the admin lockout-pause toggle (§4, intentionally resets to “lockout enabled” every restart by design).

8. Known Limitations & Roadmap

Documented as-is for transparency; not fixed as part of this document unless separately requested.

8.1 Hardcoded super-admin default credentials — RESOLVED 2026-07-11

auth/user_manager.py previously hardcoded a plaintext default password (SUPER_ADMIN_DEFAULT_PASSWORD = "ChangeMe@2025!") alongside the super-admin username, in what is currently a **public** GitHub repository. This has been fixed: seed_super_admin() now reads SUPER_ADMIN_DEFAULT_PASSWORD from the environment if set, otherwise generates a cryptographically random one-time password and logs it (once, at the moment the account is first created) so it can be captured from deployment logs. No predictable default exists in source any longer. The live skde account’s actual password was also rotated at the same time via the Admin Dashboard’s Reset Password feature, independent of this code change (the seeding function only ever affects an account at first creation, never an existing one — see the code comment at auth/user_manager.py:50-56 for detail). Note: the old literal value remains visible in git history — not remediated, since rewriting a repository’s history after multiple pushes/deploys is a disruptive operation judged not worth it here given the live password has already been rotated independently.

8.2 Password hashing inconsistency

Every account’s password (auth/user_manager.py:109-111) is hashed with unsalted hashlib.sha256 — vulnerable to rainbow-table attacks if the database is ever exposed by any means. A parallel module, auth/password_utils.py, implements salted pbkdf2_sha256 hashing correctly via passlib (already a project dependency) — but it is never actually called anywhere in the codebase, despite its own docstring claiming it exists “for consistency with user_manager.py.” This appears to be an abandoned migration. A full migration to salted hashing for all accounts is planned post-pilot.

8.3 WAL mode is not applied consistently

core/db_utils.py correctly enables PRAGMA journal_mode=WAL on every connection it opens. However, several other files open their own raw sqlite3.connect() calls without

setting this pragma — including `auth/login_security.py`, `auth/user_manager.py`, `core/admin/user_service.py`, `core/admin/system_service.py`, `core/admin/audit_logger.py`, and `core/admin/diagnostics_service.py` (all of which primarily touch `users.db`). `core/admin/data_service.py` was already fixed to route through `db_utils.py`'s connection helper for exactly this reason; the same fix should eventually be applied to the remaining files for full consistency.

8.4 `research.db` / `RESEARCH_DB_PATH` configuration mismatch

The `research.db` file is actively used — `core/analytics/llm_analysis.py` reads/writes it via a hardcoded path relative to the repo root. However, the `RESEARCH_DB_PATH` environment variable that Docker/Railway configuration sets to point at this file's intended volume location is never actually read by that code — only a standalone dev script (`smoke_test.py`) reads it. In the current deployment these happen to resolve to compatible locations, but the configuration itself is inconsistent and should be reconciled (either wire `llm_analysis.py` to read the env var, or remove the unused env var).

8.5 PostgreSQL migration path is present but non-functional

`core/db_utils.py` contains a `DB_TYPE=postgres` connection branch, but `psycpg2/psycpg2-binary` is not in `requirements.txt` — selecting this path would raise a runtime error today. This is intentional scaffolding for the post-pilot database migration, not a currently usable feature.

8.6 Miscellaneous dead/vestigial code

- `modules/registry/bootstrap.py`'s `build_registry()` is unused — the app calls `register_all_modules()` directly instead.
- `auth/password_utils.py` — see §8.2.
- The R/rpy2 integration (`streamlit_app/app.py`, `core/analytics/r_utils.py`, `launch.py`) is vestigial: no R installation exists in the Docker image, rpy2 is not an installed dependency, and the Item Response Theory analysis that once used R's `mirt` package has been fully migrated to the pure-Python `girth` library. The R-related code paths degrade gracefully (soft-fail on import) and are effectively dead outside local Windows development. A few stale docstrings in `teacher_dashboard.py` still reference “mirt”/“rpy2” and should be corrected.
- Several files (`auth/user_manager.py`, `core/db_utils.py`, `streamlit_app/app.py`) retain commented-out earlier versions of code alongside their live replacements, with terse inline notes about why the change was made (e.g. a Docker build/test failure). These should eventually be cleaned up, but carry historical context worth preserving in git history before deletion.

8.7 Dynamic dashboards (resolved — noted for history)

An earlier known limitation — the Teacher Dashboard and analytics pipeline hardcoding a fixed range of module numbers rather than consulting the module registry — has since

been resolved (`modules/registry/discover.py`'s `discover_all_module_numbers()`, adopted throughout `canonical_loader.py` and `teacher_dashboard.py`). Noted here only for continuity with earlier planning documents.

9. Document Scope Note

This document describes architecture and current state as verified directly against the live codebase on 2026-07-11. It does not cover: the post-pilot roadmap (COPPA/IRB compliance work, PostgreSQL migration, CPI+ feature enhancements — tracked separately), or line-level implementation detail better suited to the Programmer's Guide (a separate, upcoming document).